

American Put Option Pricing Using Binomial Trees, Neural Networks and Double Deep Q-Learning

Dhruv Natu

Department of Electrical Engineering
Indian Institute of Technology Bombay

July 2026

Course : Seasons of Code 2026

Project : Deep Learning for American Put Option Pricing

Repository :

<https://github.com/Dhruv271007/American-Put-Options>

Contents

1	Introduction	5
2	Methodology	6
2.1	Cox–Ross–Rubinstein (CRR) Binomial Tree	6
2.2	Neural Network Price Approximation	7
2.3	Double Deep Q-Network	8
3	Experimental Setup	10
3.1	Option Contract Parameters	10
3.2	Training Dataset	10
3.3	Neural Network Configuration	11
3.4	Double DQN Configuration	11
3.5	Hardware and Software	12
4	Results	13
4.1	Binomial Tree Validation	13
4.2	Neural Network Performance	13
4.3	Predicted versus Benchmark Prices	14
4.4	Pricing Surface Comparison	15
4.5	Pricing Error Surface	16
4.6	Double DQN Training	17
4.7	Learned Exercise Boundary	18
4.8	Policy Evaluation	19
4.9	Benchmark Comparison	19
4.10	Quantitative Performance Summary	19
5	Comparative Analysis	21
5.1	Pricing Accuracy	21
5.2	Exercise Policy Behaviour	22
5.3	Computational Characteristics	22
5.4	Training Stability	23
5.5	Overall Comparison	23
6	Limitations and Future Work	24
6.1	Binomial Tree	24
6.2	Neural Network	24
6.3	Double DQN	24
6.4	Future Work	25

7	Reproducibility	26
7.1	Repository Structure	26
7.2	Software Dependencies	26
7.3	Random Seeds	27
7.4	Running the Project	27
7.5	Repository Link	27
8	Conclusion	29

Abstract

Pricing American options differs fundamentally from pricing European options because the holder has the right to exercise the contract at any time before maturity. This transforms option pricing into an optimal stopping problem for which closed-form analytical solutions generally do not exist.

This project investigates three complementary approaches for pricing American put options. The Cox–Ross–Rubinstein (CRR) Binomial Tree is implemented as the numerical benchmark due to its financial correctness and explicit treatment of early exercise. A fully connected Neural Network is trained using supervised learning to approximate the nonlinear pricing function generated by the CRR model. Finally, a Double Deep Q-Network (Double DQN) is developed to learn the optimal stopping policy directly through reinforcement learning.

All three methods are evaluated under a common experimental framework using identical contract specifications whenever possible. Their pricing accuracy, computational efficiency, learned exercise behaviour and practical limitations are compared through quantitative metrics and visual analysis.

The results demonstrate that the Neural Network successfully approximates the benchmark pricing surface over most of the parameter space, while the Double DQN agent learns an exercise policy whose expected value closely matches the benchmark after appropriate reward engineering and hyperparameter tuning. The project illustrates both the potential and current limitations of machine learning techniques for financial derivative pricing.

1 Introduction

Financial derivatives are contracts whose values depend on underlying assets such as stocks, commodities or market indices. Among these instruments, options play a fundamental role in portfolio hedging, risk management and speculative trading.

A put option grants its holder the right, but not the obligation, to sell an underlying asset at a predetermined strike price. Unlike European options, which may only be exercised at maturity, American options allow exercise at any time before expiration. This additional flexibility significantly increases the complexity of the pricing problem because the holder must continuously decide whether exercising immediately or continuing to hold the option maximizes expected payoff.

Unlike European options, which admit the Black–Scholes closed-form solution under simplifying assumptions, American option pricing generally requires numerical methods. Classical approaches such as the Cox–Ross–Rubinstein (CRR) Binomial Tree compute the option value through backward induction while explicitly accounting for early exercise decisions. Although highly accurate, these methods become computationally expensive when pricing large collections of contracts.

Recent advances in Machine Learning provide alternative approaches to derivative pricing. Supervised learning models approximate the pricing function directly after training on benchmark prices, enabling extremely fast inference once training is complete. Reinforcement Learning approaches formulate pricing as an optimal stopping problem, allowing an agent to learn the exercise policy directly without explicit dynamic programming.

This project investigates and compares three fundamentally different methodologies:

1. Cox–Ross–Rubinstein Binomial Tree
2. Neural Network Price Approximation
3. Double Deep Q-Network Reinforcement Learning

The objective is not only to compare pricing accuracy but also computational efficiency, learned exercise behaviour and the practical strengths and weaknesses of each approach under a common evaluation framework.

2 Methodology

This project investigates three different approaches for pricing American put options. The first approach uses a classical numerical algorithm based on dynamic programming, the second formulates pricing as a supervised learning regression problem, while the third treats pricing as an optimal stopping problem solved through reinforcement learning.

Each method is implemented independently and evaluated under a common experimental framework.

2.1 Cox–Ross–Rubinstein (CRR) Binomial Tree

The Cox–Ross–Rubinstein (CRR) Binomial Tree provides the benchmark solution throughout this project. It approximates the continuous evolution of the stock price using a recombining discrete-time lattice.

For a time discretisation consisting of N steps, the stock price evolves according to

$$S_{t+\Delta t} = \begin{cases} uS_t, & \text{with probability } p, \\ dS_t, & \text{with probability } 1 - p, \end{cases}$$

where

$$u = e^{\sigma\sqrt{\Delta t}},$$

$$d = \frac{1}{u},$$

and

$$\Delta t = \frac{T}{N}.$$

The corresponding risk-neutral probability is

$$p = \frac{e^{r\Delta t} - d}{u - d}.$$

For an American put option, the value at each node is computed using backward induction. At every node the continuation value is compared against the immediate exercise payoff,

$$K - S.$$

Hence the option value satisfies

$$V = \max \left(K - S, e^{-r\Delta t} (pV_u + (1 - p)V_d) \right).$$

This formulation naturally captures the early exercise feature and therefore serves as the numerical benchmark for both machine learning approaches.

2.2 Neural Network Price Approximation

The second approach formulates option pricing as a supervised regression problem.

Instead of solving the pricing recursion for every new option contract, a neural network is trained to approximate the nonlinear mapping

$$(S_0, K, T, r, \sigma) \longrightarrow V.$$

Training labels are generated using the CRR Binomial Tree over a large collection of randomly sampled option contracts.

The network receives five input features

- Initial stock price (S_0)
- Strike price (K)
- Time to maturity (T)
- Risk-free interest rate (r)
- Volatility (σ)

and predicts a single scalar corresponding to the American put price.

Prior to training, both the input features and target prices are standardised using

$$x' = \frac{x - \mu}{\sigma}.$$

Standardising the target values significantly improved numerical stability during optimisation and prevented large gradients caused by differing option price scales.

The neural network consists of fully connected layers with ReLU activation functions.

$$5 \rightarrow 128 \rightarrow 128 \rightarrow 1.$$

The final output layer is linear since the problem is formulated as a regression task.

Training is performed using the Adam optimiser together with Mean Squared Error (MSE) loss.

2.3 Double Deep Q-Network

Unlike supervised learning, reinforcement learning does not directly predict option prices.

Instead, the pricing problem is formulated as an optimal stopping problem where an agent learns when exercising the option maximises expected discounted payoff.

The environment state is represented as

$$(\tau, S/K, t/T),$$

where

- current timestep,
- moneyness,
- remaining time fraction

describe the current contract state.

At every timestep the agent selects one of two possible actions

- Hold
- Exercise

If the agent exercises, the episode terminates immediately and receives the intrinsic payoff

$$\max(K - S, 0).$$

If the agent chooses to hold, the stock price evolves according to the CRR transition probabilities and the episode continues until either exercise or maturity.

Experience replay is employed to reduce correlation between successive samples, while a separate target network stabilises learning.

To reduce the overestimation bias commonly observed in Deep Q-Networks, the standard DQN algorithm was replaced by Double DQN.

The online network selects the greedy action,

$$a^* = \arg \max_a Q_{\text{online}}(s', a),$$

while the target network evaluates this action,

$$y = r + \gamma Q_{\text{target}}(s', a^*).$$

This decoupling significantly improves training stability and produces more reliable value estimates.

Furthermore, rewards were normalised by the strike price during training. This simple reward engineering step substantially improved convergence and prevented instability caused by large payoff magnitudes.

3 Experimental Setup

To enable a fair comparison between the numerical benchmark, supervised learning model and reinforcement learning agent, all experiments were conducted under a common set of option contract specifications whenever applicable. Random seeds were fixed to ensure reproducibility and identical market parameters were used across all methods.

3.1 Option Contract Parameters

The primary contract used for benchmarking the reinforcement learning agent is summarised in Table 1.

Table 1: Option Contract Parameters

Parameter	Value
Initial Stock Price (S_0)	100
Strike Price (K)	100
Risk-free Rate (r)	0.05
Volatility (σ)	0.20
Time to Maturity (T)	1 year
Binomial Steps (N)	25
Option Type	American Put

3.2 Training Dataset

The supervised learning dataset was generated using the CRR Binomial Tree by randomly sampling option contracts over a broad parameter space. Each sampled contract was priced using the benchmark implementation and used as a labelled training example.

The sampled ranges are shown in Table 2.

Table 2: Training Data Generation

Parameter	Sampling Range
Stock Price (S_0)	25 – 175
Strike Price (K)	$0.75 \times S_0 - 1.25 \times S_0$
Volatility (σ)	0.05 – 0.40
Time to Maturity (T)	0.10 – 2.00 years
Risk-free Rate (r)	0.01 – 0.15

Prior to training, both the feature vectors and target option prices were standardised using the statistics computed from the training set. The same transformation was subsequently applied to the validation and test sets to prevent information leakage.

3.3 Neural Network Configuration

The neural network architecture and optimisation parameters are summarised in Table 3.

Table 3: Neural Network Hyperparameters

Hyperparameter	Value
Architecture	5-64-16-1
Activation Function	ReLU
Output Layer	Linear
Loss Function	Mean Squared Error
Optimizer	Adam
Learning Rate	0.01
Epochs	500
Feature Scaling	StandardScaler
Target Scaling	StandardScaler

3.4 Double DQN Configuration

The reinforcement learning agent was trained using experience replay together with a target network. Hyperparameters were tuned empirically to improve convergence while maintaining stable learning.

Table 4 summarises the final configuration.

Table 4: Double DQN Hyperparameters

Hyperparameter	Value
Episodes	15000
Replay Buffer Size	50000
Batch Size	128
Learning Rate	0.01
Discount Factor (γ)	0.99
Target Network Update	Every 50 Updates
Initial ϵ	1.0
Minimum ϵ	0.01
ϵ Decay	0.999
Gradient Clipping	5.0
Reward Normalisation	Payoff / Strike Price
Random Seed	42

The target network parameters were periodically synchronised with the online network every 50 optimisation steps to improve stability. Experience replay was used to reduce sample correlation and improve data efficiency.

3.5 Hardware and Software

All experiments were implemented in Python using the PyTorch deep learning framework.

Training was performed on an NVIDIA RTX 5060 Laptop GPU with CUDA acceleration enabled for both the neural network and reinforcement learning implementations.

The principal software libraries used include:

- PyTorch
- NumPy
- Pandas
- Matplotlib
- Scikit-Learn

Random seeds were fixed throughout the project to improve reproducibility.

4 Results

This section presents the performance of each implemented methodology under the common experimental framework described previously. The Cox–Ross–Rubinstein (CRR) Binomial Tree serves as the benchmark against which both the Neural Network and the Double DQN reinforcement learning agent are evaluated.

4.1 Binomial Tree Validation

The CRR implementation was first validated independently before being used as the benchmark for subsequent experiments. The pricing algorithm correctly captures the early exercise feature of American put options through backward induction.

Figure 1 illustrates the convergence of the American put option price as the number of binomial timesteps increases.

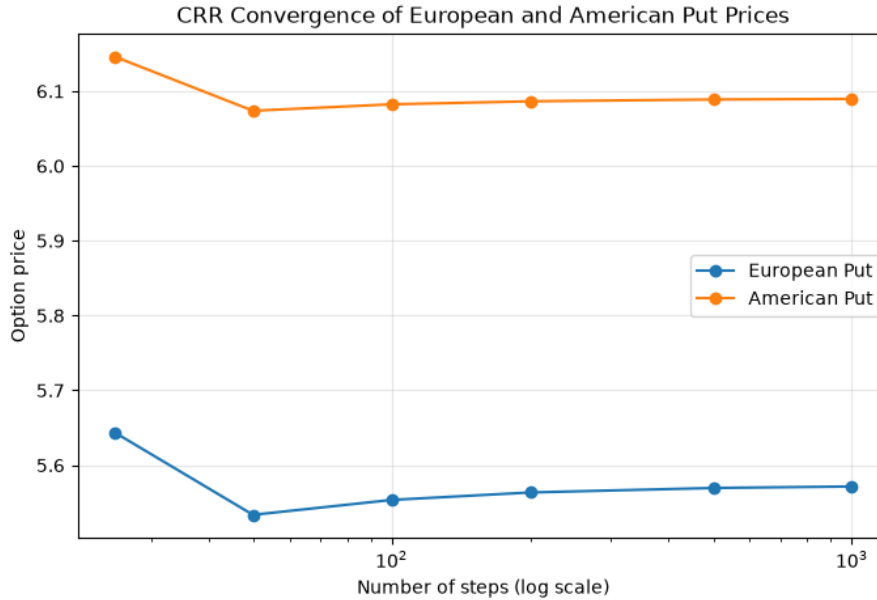


Figure 1: Convergence of the CRR Binomial Tree price with increasing number of timesteps.

As expected, the estimated option value stabilises as the number of timesteps increases, demonstrating numerical convergence of the benchmark implementation.

4.2 Neural Network Performance

The supervised learning model was trained using CRR-generated option prices as regression targets.

Figure 2 shows the evolution of both training and validation losses during optimisation.

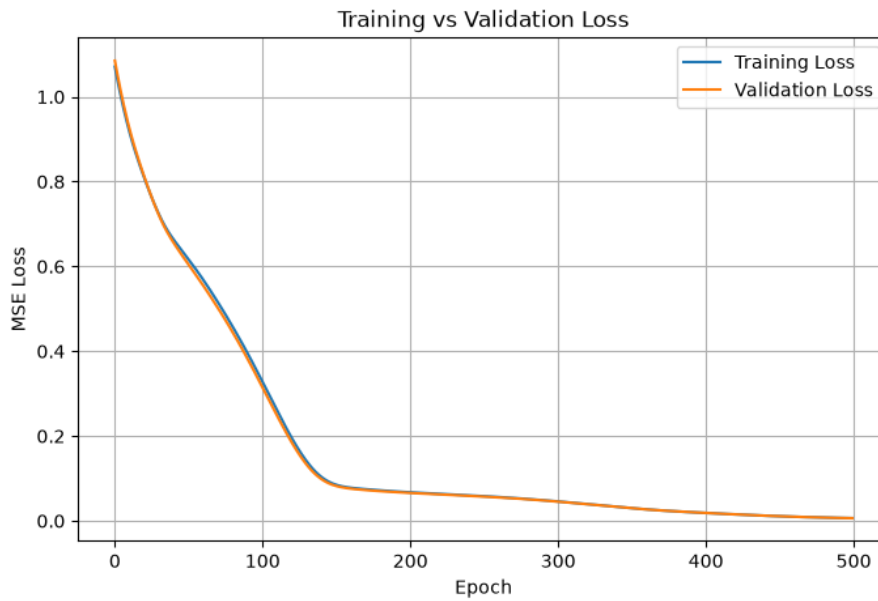


Figure 2: Training and validation loss during neural network optimisation.

Both losses decrease steadily throughout training with no significant divergence between the training and validation curves, indicating that the model generalises well over the sampled parameter space.

4.3 Predicted versus Benchmark Prices

To evaluate pricing accuracy, the neural network predictions were compared against the benchmark CRR prices on the held-out test dataset.

Figure 3 presents the predicted-versus-true scatter plot.

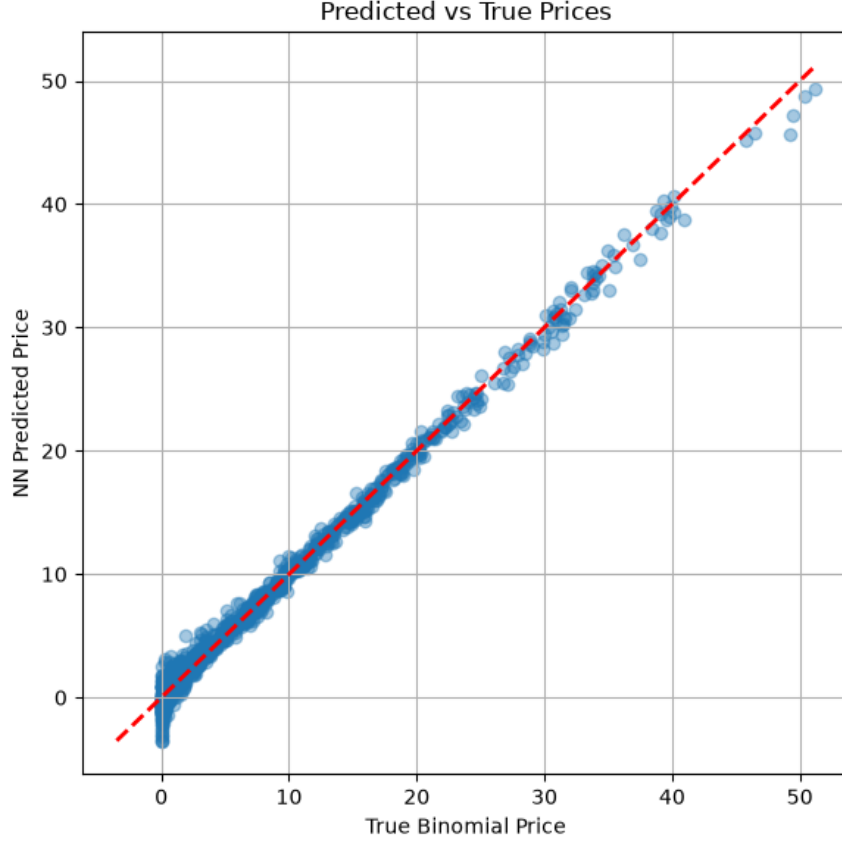


Figure 3: Neural network predictions compared with benchmark CRR prices.

Most predictions lie close to the diagonal reference line, indicating that the neural network successfully captures the nonlinear pricing function over the majority of the contract space.

The neural network nevertheless exhibits larger prediction errors for deep in-the-money contracts where comparatively fewer training samples are available.

4.4 Pricing Surface Comparison

A qualitative comparison of the learned pricing surface was performed by evaluating both the CRR model and the neural network over a dense grid of stock prices and maturities.

Figure 4 compares the two pricing surfaces.

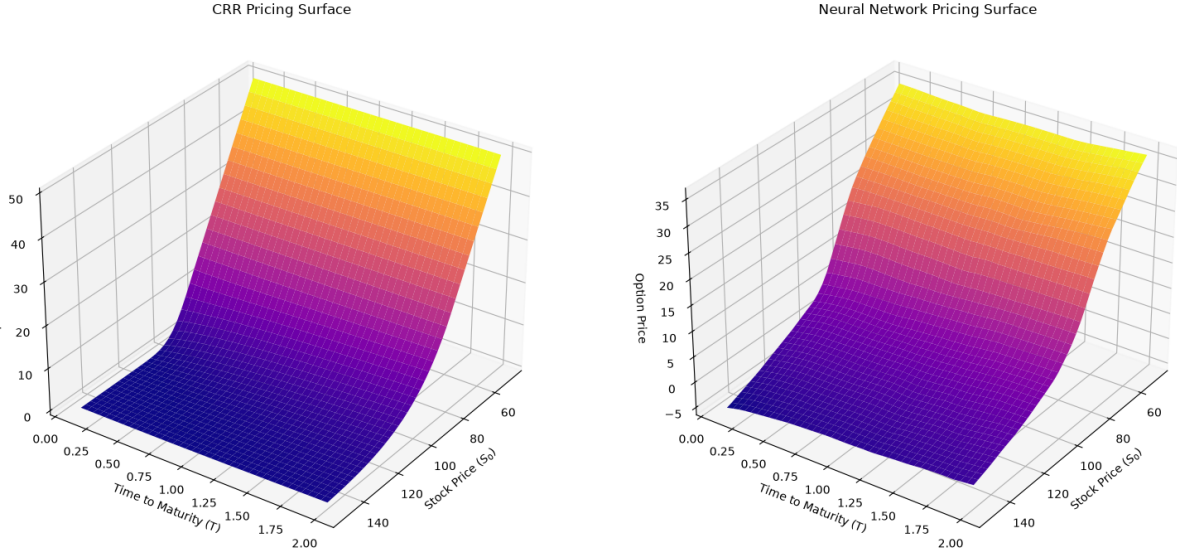


Figure 4: Comparison of CRR pricing surface and Neural Network approximation.

The neural network successfully reproduces the overall shape of the benchmark pricing surface, including the monotonic increase in option value as the stock price decreases.

Some underestimation remains visible in the deep in-the-money region where the benchmark prices approach the intrinsic value.

4.5 Pricing Error Surface

To better understand the regions where the neural network deviates from the benchmark, the absolute pricing error

$$|V_{NN} - V_{CRR}|$$

was evaluated over the same parameter grid.

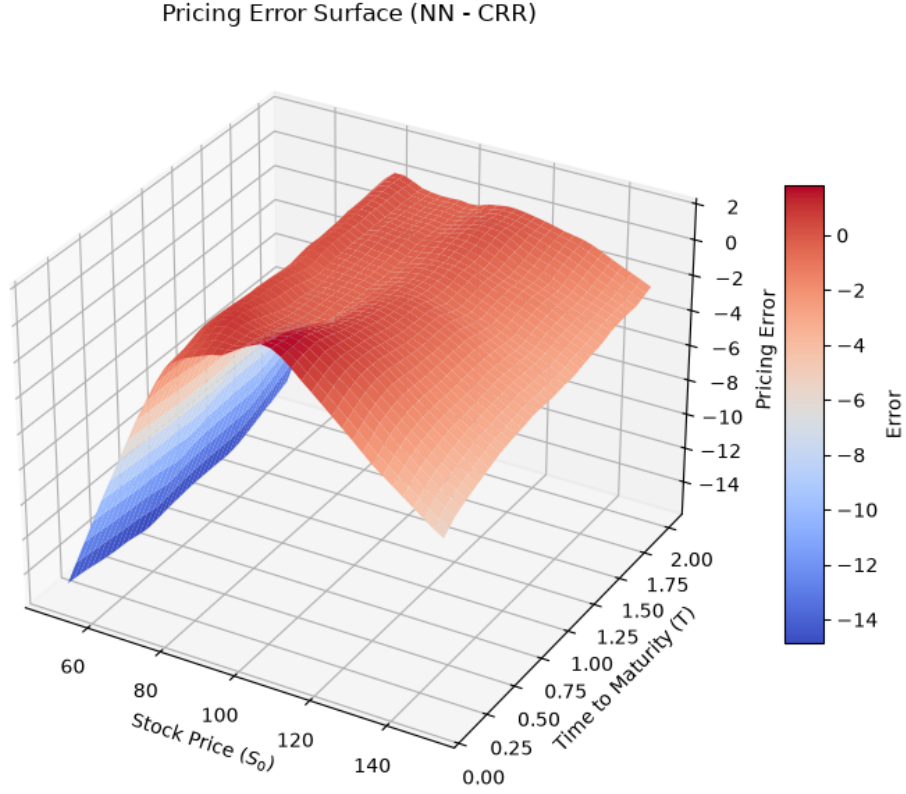


Figure 5: Absolute pricing error between the Neural Network and CRR benchmark.

The largest deviations occur primarily for deep in-the-money contracts, while the approximation remains highly accurate throughout the majority of the parameter domain.

4.6 Double DQN Training

The reinforcement learning agent was trained using experience replay together with a periodically updated target network.

Figure 6 shows the training loss throughout optimisation.

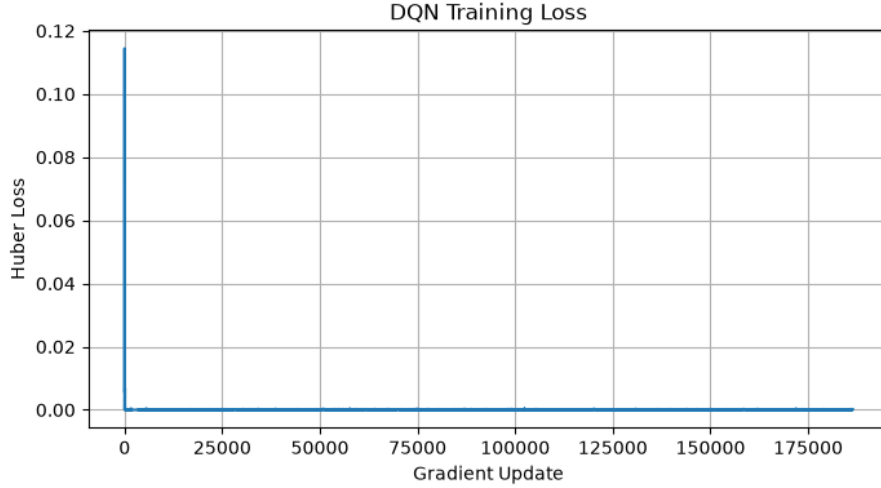


Figure 6: Double DQN training loss.

Reward normalisation by the strike price significantly improved convergence and reduced instability compared to the initial implementation.

4.7 Learned Exercise Boundary

One of the primary objectives of the reinforcement learning agent is to learn the optimal early exercise policy.

Figure 7 illustrates the learned stopping boundary.

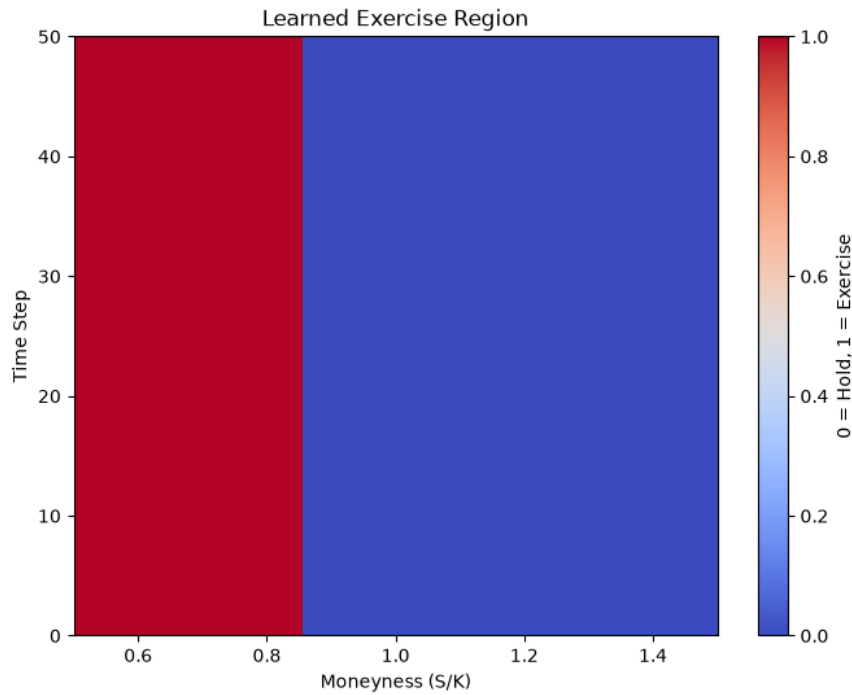


Figure 7: Exercise region learned by the Double DQN agent.

The learned policy exercises predominantly in deep in-the-money regions while con-

tinuing to hold for contracts with substantial continuation value. This behaviour is consistent with financial intuition for American put options.

4.8 Policy Evaluation

The learned policy was evaluated against several baseline stopping strategies.

Table 5: Policy Evaluation

Policy	Value	Exercise Rate	Avg Exercise Step
Always Hold	5.6390	0.0000	—
Immediate Exercise	0.0000	1.0000	0.0
Random	1.0531	1.0000	1.03
Double DQN	6.0943	0.3568	12.29

The Double DQN policy produces the highest expected discounted payoff while exercising only when continuation is no longer advantageous.

4.9 Benchmark Comparison

For the evaluation contract

$$S_0 = 100, K = 100, r = 0.05, \sigma = 0.20, T = 1, N = 25,$$

the benchmark and reinforcement learning values are

$$V_{CRR} = 6.146185,$$

$$V_{DoubleDQN} = 6.0943.$$

The resulting absolute pricing error is

$$|6.146185 - 6.0943| = 0.05187,$$

which demonstrates that the learned stopping policy closely approximates the optimal benchmark solution.

4.10 Quantitative Performance Summary

The overall performance of the three approaches is summarised in Table 6. The CRR Binomial Tree serves as the benchmark, while the Neural Network and Double DQN are evaluated relative to it.

Table 6: Summary of Model Performance

Metric	CRR	Neural Network	Double DQN
Pricing Accuracy	Benchmark	RMSE = RMSE : 0.791919	Error = 0.05187
Inference Speed	Moderate	Very Fast	Fast
Training Required	No	Yes	Yes
Early Exercise	Exact	Indirect	Learned
Generalisation	Exact	Good	Good

5 Comparative Analysis

The three approaches investigated in this project solve the American put option pricing problem using fundamentally different methodologies. While the CRR Binomial Tree computes the exact optimal stopping value through dynamic programming, the Neural Network learns a nonlinear approximation of the pricing function using supervised learning, and the Double Deep Q-Network (Double DQN) learns the optimal stopping strategy through interaction with the pricing environment.

Each approach possesses distinct advantages and limitations, which are discussed below.

5.1 Pricing Accuracy

The CRR Binomial Tree serves as the numerical benchmark throughout this project. Since every option value is computed through backward induction while explicitly considering early exercise opportunities, it represents the reference against which all machine learning models are evaluated.

The Neural Network successfully approximates the highly nonlinear pricing surface over most of the parameter domain. Training and validation losses decrease consistently, demonstrating stable optimisation without significant overfitting. The predicted-versus-benchmark scatter plot further shows that the majority of predictions lie close to the ideal diagonal.

Nevertheless, larger approximation errors remain visible in the deep in-the-money region. These contracts occur relatively infrequently within the randomly generated dataset, causing the regression model to receive comparatively fewer representative training examples. As a result, the network slightly underestimates prices in these regions despite accurately modelling the remainder of the pricing surface.

Unlike the Neural Network, the Double DQN agent does not explicitly predict option prices. Instead, it learns an exercise policy whose expected discounted payoff is compared against the benchmark.

For the benchmark contract

$$S_0 = 100, K = 100, r = 0.05, \sigma = 0.20, T = 1, N = 25,$$

the obtained values are

$$V_{\text{CRR}} = 6.146185,$$

and

$$V_{\text{DoubleDQN}} = 6.0943.$$

The absolute difference

$$|V_{\text{DoubleDQN}} - V_{\text{CRR}}| = 0.051866209909707806$$

is extremely small, indicating that the learned stopping policy closely approximates the benchmark solution.

5.2 Exercise Policy Behaviour

One of the principal objectives of reinforcement learning is to discover the optimal stopping policy rather than directly approximate option prices.

The learned exercise boundary demonstrates behaviour that is broadly consistent with financial intuition. Contracts that are deeply in-the-money are exercised more frequently, whereas contracts possessing substantial continuation value are generally held until later timesteps.

The policy evaluation further supports this conclusion.

Compared with the baseline strategies,

- Immediate Exercise discards nearly all time value.
- Always Hold ignores valuable early exercise opportunities.
- Random Exercise performs poorly due to arbitrary stopping decisions.
- Double DQN balances continuation and exercise decisions, producing the highest expected discounted payoff.

This behaviour indicates that the reinforcement learning agent has successfully learned the fundamental trade-off between intrinsic value and continuation value.

5.3 Computational Characteristics

The computational behaviour of the three approaches differs considerably.

The CRR Binomial Tree performs an explicit dynamic programming recursion for every contract individually. Although highly accurate, computational cost increases approximately quadratically with the number of timesteps.

The Neural Network requires a relatively expensive offline training phase. However, once trained, inference consists of a single forward pass through the network, allowing thousands of option prices to be generated almost instantaneously.

Similarly, the Double DQN agent requires substantial training time because it must repeatedly interact with the environment while updating the Q-network. Once training is complete, however, policy inference is computationally inexpensive and requires only a forward pass through the learned network.

5.4 Training Stability

Several implementation improvements significantly increased the stability of both machine learning models.

For the supervised learning model, standardising both the input features and target option prices prevented large gradients and accelerated convergence during optimisation.

For reinforcement learning, the original Deep Q-Network exhibited unstable value estimates and inconsistent exercise behaviour. Replacing it with Double DQN reduced overestimation bias by separating action selection from action evaluation.

An additional improvement was obtained by normalising rewards with respect to the strike price,

$$R = \frac{\max(K - S, 0)}{K},$$

which substantially reduced variance in Q-value targets and produced a significantly more stable learning process.

5.5 Overall Comparison

Table 7 summarises the qualitative strengths of each methodology.

Table 7: Comparison of Pricing Approaches

Property	CRR	Neural Network	Double DQN
Pricing Accuracy	Excellent	Very Good	Good
Early Exercise	Exact	Learned Indirectly	Learned Directly
Training Required	No	Yes	Yes
Inference Speed	Moderate	Very Fast	Fast
Interpretability	High	Moderate	Moderate
Scalability	Moderate	Excellent	Excellent

Overall, the CRR Binomial Tree remains the most reliable approach for obtaining benchmark prices, while the Neural Network offers an efficient approximation once trained. The Double DQN formulation provides the additional capability of directly learning an optimal stopping strategy and produces exercise decisions that closely resemble the benchmark policy.

6 Limitations and Future Work

Although the implemented methods successfully demonstrate multiple approaches to pricing American put options, several limitations remain that provide opportunities for future improvement.

6.1 Binomial Tree

The Cox–Ross–Rubinstein Binomial Tree provides highly accurate benchmark prices for low-dimensional contracts. However, computational complexity increases rapidly as the number of timesteps grows since every additional timestep increases the number of nodes that must be evaluated during backward induction.

While suitable for generating benchmark prices and training labels, the method becomes computationally expensive when pricing extremely large numbers of contracts or higher-dimensional derivatives.

6.2 Neural Network

The supervised learning approach successfully approximates the benchmark pricing surface but remains dependent on the quality and diversity of the generated training dataset.

The largest prediction errors occur in deep in-the-money regions where comparatively fewer samples are available. Since the network learns only from examples observed during training, its performance naturally deteriorates in underrepresented regions of the parameter space.

Another limitation arises from the unconstrained linear output layer, which may occasionally produce small negative option prices for contracts whose true values are very close to zero. Although these predictions are rare and usually of negligible magnitude, incorporating a non-negative output constraint or predicting transformed option prices could further improve robustness.

6.3 Double DQN

The reinforcement learning formulation introduces additional challenges compared to supervised learning.

Training is considerably more computationally expensive because the agent must repeatedly interact with the environment before discovering an effective exercise strategy. Performance is also sensitive to hyperparameter selection, including the learning rate, replay buffer size, exploration schedule and target network update frequency.

Early experiments exhibited unstable learning behaviour and excessive overestimation of option values. Replacing the original Deep Q-Network with Double DQN together with

reward normalisation significantly improved convergence and produced exercise policies much closer to the benchmark solution.

Although the learned stopping boundary closely resembles financial intuition, further improvements could likely be achieved using more advanced reinforcement learning algorithms such as Dueling DQN, Prioritized Experience Replay or distributional reinforcement learning.

6.4 Future Work

Several natural extensions of this project remain possible.

Future work could include pricing higher-dimensional derivatives, incorporating stochastic volatility models such as the Heston model, or extending the reinforcement learning framework to basket options and multi-asset derivatives.

For the supervised learning model, increasing the size and diversity of the training dataset together with more sophisticated architectures may improve approximation accuracy, particularly for extreme market conditions.

The reinforcement learning formulation could also benefit from curriculum learning, improved state representations and advanced exploration strategies to further accelerate convergence and improve policy quality.

Overall, this project demonstrates that modern machine learning techniques provide promising alternatives to traditional numerical methods while also highlighting the importance of careful data generation, reward engineering and algorithm selection for reliable financial applications.

7 Reproducibility

A major objective of this project was to ensure that all experiments are fully reproducible. The complete source code, trained models, generated figures and report are available through the accompanying GitHub repository.

7.1 Repository Structure

The repository is organised into independent modules corresponding to each stage of the project:

- **Week 4:** Cox–Ross–Rubinstein (CRR) Binomial Tree implementation.
- **Week 6:** Neural Network dataset generation, training and evaluation.
- **Week 7–8:** Reinforcement Learning environment, Double Deep Q-Network implementation and policy evaluation.
- **Week 9:** Final comparison, visualisations and report generation.

Each notebook can be executed independently while sharing the common pricing framework.

7.2 Software Dependencies

The project was implemented entirely in Python using the following major libraries:

- Python 3.x
- PyTorch
- NumPy
- Pandas
- Matplotlib
- Scikit-Learn

A complete list of package versions is provided in the accompanying `requirements.txt` file.

7.3 Random Seeds

To improve reproducibility, all stochastic components were executed using fixed random seeds. This includes dataset generation, reinforcement learning environment initialisation and neural network training wherever applicable.

The experiments reported in this work were performed using

Random Seed = 42.

7.4 Running the Project

The accompanying repository contains a detailed README describing project setup, installation and execution instructions.

The workflow consists of

1. Install the required dependencies.
2. Generate benchmark prices using the CRR Binomial Tree.
3. Train the Neural Network pricing model.
4. Train the Double DQN reinforcement learning agent.
5. Evaluate all methods under the common benchmark contracts.
6. Generate pricing surfaces, exercise boundary plots and comparison figures.

The repository also contains all figures included in this report together with the trained model checkpoints.

7.5 Repository Link

The complete project is publicly available at

<https://github.com/Dhruv271007/American-Put-Options>

The repository includes

- Source code
- Trained models
- Figures
- README

- Requirements file
- Final report

allowing the complete experimental pipeline to be reproduced from scratch.

8 Conclusion

This project covered three fundamentally different approaches for pricing American put options: the Cox–Ross–Rubinstein (CRR) Binomial Tree Model, a supervised Neural Network, and a Double Deep Q-Network (Double DQN) reinforcement learning agent.

The CRR Binomial Tree served as the numerical benchmark throughout the project due to its ability to explicitly account for the early exercise feature of American options. Although computationally more expensive than machine learning approaches, it consistently provides financially correct prices and remains the reference standard for comparison.

The supervised Neural Network demonstrated that American option prices can be accurately approximated using regression on CRR-generated data. After standardising both the input features and target prices, the network converged reliably and successfully reproduced the overall pricing surface. The trained model achieved significantly faster inference than the benchmark while maintaining good pricing accuracy across most of the parameter space. Prediction errors were concentrated in deep in the money and deep out of the money regions, highlighting the importance of representative training data.

The reinforcement learning formulation approached the problem from a different perspective by learning the optimal stopping policy directly through interaction with the pricing environment. Replacing the original Deep Q-Network with Double DQN, together with reward normalisation and careful hyperparameter tuning, resulted in substantially improved training stability. The learned policy produced economically sensible exercise decisions and achieved an expected option value of approximately 6.0943, closely matching the CRR benchmark value of 6.146185 for the evaluation contract.

Overall, this project demonstrates that classical numerical finance and modern machine learning provide complementary solutions to American option pricing. The CRR Binomial Tree offers highly accurate benchmark prices, supervised learning enables rapid approximation of the pricing function after training, and reinforcement learning provides a practical framework for learning optimal early exercise policies without explicitly solving the dynamic programming recursion.